

■ IRON LINKS GYM

Tarea #3 — Reto Final

Memoria Técnica

Alumno: Dan Bolocan

Módulo: Bases de Datos — PL/SQL

Ciclo: 1.º DAW · CPIFP Los Enlaces

Caso de uso: Gimnasio con socios, tarifas y clases

Fecha: 14/05/2026

Índice

1. Visión general del caso de uso
2. Modelo Entidad-Relación (E/R)
3. Modelo relacional y normalización
4. Descripción de tablas y restricciones
5. Datos de ejemplo
6. Batería de consultas SQL
7. PL/SQL: procedimientos, funciones y cursores
8. Triggers
9. Conclusión

1. Visión general del caso de uso

El caso de uso elegido es Iron Links Gym, un gimnasio moderno que necesita gestionar socios, tarifas de abono, contratos, pagos, clases grupales y la disponibilidad de sus instalaciones. La base de datos cubre el ciclo de vida completo del socio: alta, contratación de tarifa, inscripción a clases, seguimiento de asistencia, renovación y baja.

Este dominio resulta especialmente rico para PL/SQL porque combina reglas de negocio que deben aplicarse de forma automática (control de aforo, cálculo de fecha de vencimiento, auditoría RGPD) con operaciones transaccionales complejas (renovación de contratos, inscripciones condicionadas a la vigencia del abono).

Entidades principales:

Entidad	Rol en el negocio
Sala	Espacio físico donde se imparten las clases. Tiene aforo máximo.
Monitor	Profesional que imparte las clases. Tiene especialidad y contacto.
Tarifa	Plan de abono (mensual, trimestral, anual...) con precio y duración.
Socio	Cliente del gimnasio. Tiene NIF único, estado y fecha de alta.
Contrato	Vincula un socio con una tarifa en un período de tiempo concreto.
Clase	Sesión grupal con sala, monitor, horario y límite de plazas.
Inscripcion_clase	Relación N:M entre Socio y Clase. Registra fecha y asistencia.
Pago_cuota	Registro de cada cobro asociado a un contrato.
Auditoria_socios	Historial de cambios de estado en la entidad Socio (RGPD). Sin FK real hacia Socio para preservar datos históricos aunque el socio sea eliminado.

2. Modelo Entidad-Relación (E/R)

A continuación se describe el diagrama E/R. Las relaciones se representan indicando cardinalidad, entidades participantes y atributos propios de la relación.

Relación	Entidades	Cardinalidad	Atributos propios
TIENE	SALA – CLASE	1 : N	—
IMPARTE	MONITOR – CLASE	1 : N	—
CONTRATA	SOCIO – TARIFA	N : M	fecha_inicio, fecha_fin, importe_pag, estado (resuelta por CONTRATO)
SE_INSCRIBE	SOCIO – CLASE	N : M	fecha_insc, asistio (resuelta por INSCRIPCION_CLASE)
GENERA	CONTRATO – PAGO_CUOTA	1 : N	fecha_pago, importe, metodo_pago
AUDITA	SOCIO → AUDITORIA_SOCIOS	1 : N (lógica)	accion, campo, valor_antes, valor_despues. Sin FK real: los registros se conservan aunque el socio sea eliminado.

Justificación de las relaciones N:M

SOCIO – TARIFA (CONTRATO): un socio puede contratar varias tarifas a lo largo del tiempo (renovaciones) y una tarifa puede ser contratada por muchos socios. La tabla intermedia CONTRATO añade los atributos temporales y económicos de cada vinculación.

SOCIO – CLASE (INSCRIPCION_CLASE): un socio puede inscribirse en muchas clases y una clase puede tener muchos socios. La tabla intermedia registra cuándo se inscribió y si finalmente asistió, datos que no pertenecen ni al socio ni a la clase por separado.

3. Modelo relacional y normalización

Todas las tablas se encuentran en 3.ª Forma Normal (3FN). A continuación se indica el esquema relacional y la justificación de las claves primarias elegidas.

Tabla	Clave primaria	Justificación
SALA	PK: id_sala	Clave sintética porque el nombre de la sala puede cambiar.
MONITOR	PK: id_monitor	Email y teléfono son UNIQUE como identificación natural alternativa.
TARIFA	PK: id_tarifa	El nombre puede modificarse; la PK no debe cambiar para conservar FK en contratos.
SOCIO	PK: id_socio	NIF marcado UNIQUE como clave natural alternativa. Email también UNIQUE.
CONTRATO	PK: id_contrato	Un socio puede tener múltiples contratos; la PK distingue cada período.
CLASE	PK: id_clase	Nombre+día+hora no es suficientemente estable como PK natural.
INSCRIPCION_CLASE	PK: id_inscripcion	Constraint UNIQUE(id_socio, id_clase) evita duplicados; PK sintética para FK futuras.
PAGO_CUOTA	PK: id_pago	Cada pago es un evento independiente incluso del mismo contrato.
AUDITORIA_SOCIOS	PK: id_auditoria	Tabla de solo inserción; nunca se borran registros de auditoría. Sin FK hacia SOCIO.

Verificación de Formas Normales

- 1FN: todos los atributos son atómicos. No hay grupos repetidos ni arrays.
- 2FN: no hay dependencias parciales. En INSCRIPCION_CLASE, los atributos fecha_insc y asistio dependen de la clave completa (id_socio, id_clase), no de una parte de ella.
- 3FN: no hay dependencias transitivas. Por ejemplo, el precio de la tarifa está en TARIFA, no en CONTRATO (donde se copia como importe_pag para preservar el valor histórico sin depender transitivamente de TARIFA).

4. Descripción de tablas y restricciones

SALA

Columna	Tipo	Restricción	Notas
id_sala	NUMBER	PK (seq_sala)	
nombre	VARCHAR2(60)	NOT NULL	
aforo_max	NUMBER(3)	NOT NULL, CHECK (1..200)	
descripcion	VARCHAR2(200)	—	

MONITOR

Columna	Tipo	Restricción	Notas
id_monitor	NUMBER	PK	
nombre	VARCHAR2(80)	NOT NULL	
apellidos	VARCHAR2(100)	NOT NULL	
especialidad	VARCHAR2(60)	—	
telefono	VARCHAR2(15)	UNIQUE	Identificación natural
email	VARCHAR2(100)	UNIQUE	Identificación natural
fecha_alta	DATE	NOT NULL, DEFAULT SYSDATE	

TARIFA

Columna	Tipo	Restricción	Notas
id_tarifa	NUMBER	PK	
nombre	VARCHAR2(60)	NOT NULL	
duracion_dias	NUMBER(4)	NOT NULL, CHECK > 0	
precio	NUMBER(8,2)	NOT NULL, CHECK > 0	
descripcion	VARCHAR2(200)	—	
activa	CHAR(1)	NOT NULL, CHECK IN ('S','N')	

SOCIO

Columna	Tipo	Restricción	Notas
id_socio	NUMBER	PK	
nif	VARCHAR2(10)	NOT NULL, UNIQUE	Clave natural alternativa

Columna	Tipo	Restricción	Notas
nombre	VARCHAR2(80)	NOT NULL	
apellidos	VARCHAR2(100)	NOT NULL	
fecha_nac	DATE	NOT NULL	Validada por trigger (≥ 16 años)
telefono	VARCHAR2(15)	—	
email	VARCHAR2(100)	UNIQUE	
fecha_alta	DATE	NOT NULL, DEFAULT SYSDATE	
estado	VARCHAR2(10)	NOT NULL, CHECK IN ('ACTIVO','BAJA','SUSPENDIDO')	

CONTRATO

Columna	Tipo	Restricción	Notas
id_contrato	NUMBER	PK	
id_socio	NUMBER	FK SOCIO, NOT NULL	
id_tarifa	NUMBER	FK TARIFA, NOT NULL	
fecha_inicio	DATE	NOT NULL	
fecha_fin	DATE	—	Calculada automáticamente por trigger
importe_pag	NUMBER(8,2)	—	Precio real pagado (puede diferir de tarifa)
estado	VARCHAR2(10)	NOT NULL, CHECK IN ('ACTIVO','VENCIDO','CANCELADO')	

CLASE

Columna	Tipo	Restricción	Notas
id_clase	NUMBER	PK	
nombre	VARCHAR2(80)	NOT NULL	
id_sala	NUMBER	FK SALA, NOT NULL	
id_monitor	NUMBER	FK MONITOR, NOT NULL	
dia_semana	VARCHAR2(10)	NOT NULL, CHECK (LUNES..DOMINGO)	
hora_inicio	VARCHAR2(5)	NOT NULL	
duracion_min	NUMBER(3)	NOT NULL, CHECK (15..240)	
plazas_max	NUMBER(3)	NOT NULL, CHECK > 0	
plazas_libres	NUMBER(3)	NOT NULL, CHECK $\geq$	Mantenido por triggers

Columna	Tipo	Restricción	Notas
		0	
activa	CHAR(1)	NOT NULL, CHECK IN ('S','N')	

INSCRIPCION_CLASE

Columna	Tipo	Restricción	Notas
id_inscripcion	NUMBER	PK	
id_socio	NUMBER	FK SOCIO, NOT NULL	
id_clase	NUMBER	FK CLASE, NOT NULL	
fecha_insc	DATE	NOT NULL, DEFAULT SYSDATE	
asistio	CHAR(1)	NOT NULL, CHECK IN ('P','S','N'); UNIQUE(id_socio, id_clase)	P=Pendiente, S=Sí, N=No

PAGO_CUOTA

Columna	Tipo	Restricción	Notas
id_pago	NUMBER	PK	
id_contrato	NUMBER	FK CONTRATO, NOT NULL	
fecha_pago	DATE	NOT NULL, DEFAULT SYSDATE	
importe	NUMBER(8,2)	NOT NULL, CHECK > 0	
metodo_pago	VARCHAR2(20)	NOT NULL, CHECK IN (DOMICILIACION, TARJETA, EFECTIVO, TRANSFERENCIA)	
observaciones	VARCHAR2(200)	—	

AUDITORIA_SOCIOS

Columna	Tipo	Restricción	Notas
id_auditoria	NUMBER	PK	
id_socio	NUMBER	NOT NULL	Sin FK: conserva histórico si el socio es eliminado
accion	VARCHAR2(10)	NOT NULL, CHECK IN ('INSERT','UPDATE','DELETE')	
campo	VARCHAR2(60)	—	

Columna	Tipo	Restricción	Notas
valor_antes	VARCHAR2(200)	—	
valor_despues	VARCHAR2(200)	—	
usuario_bd	VARCHAR2(60)	DEFAULT USER	
fecha_cambio	DATE	NOT NULL, DEFAULT SYSDATE	

5. Datos de ejemplo (insertar.sql)

Se han insertado datos realistas que permiten probar todos los casos de las consultas y de la lógica PL/SQL. El volumen por tabla es:

Tabla	Filas	Observaciones
SALA	6	Cardio, Spinning, Musculación, 2 Polivalentes, Piscina
MONITOR	6	Especialidades variadas: yoga, spinning, natación, kickboxing...
TARIFA	8	Mensual/Trimestral/Anual × Básica/Completa + Pase Día + Jubilados
SOCIO	15	Incluye estados ACTIVO, BAJA y SUSPENDIDO para probar filtros
CONTRATO	16	Incluye estados ACTIVO y VENCIDO; renovación del socio 3
CLASE	11	Distribuidas en distintos días y salas
INSCRIPCION_CLASE	20	Mezcla de asistencias P, S y N. Requiere aplicar la corrección del trigger trg_no_solapamiento_clase antes de insertar (ver sección 8).
PAGO_CUOTA	16	Un pago por contrato + uno de renovación (contrato 16)

Nota sobre las inscripciones:

Las inscripciones en INSCRIPCION_CLASE no se insertan directamente con el fichero insertar.sql original debido a un error de tabla mutante (ORA-04091) provocado por la cadena de triggers trg_plazas_inscripcion y trg_no_solapamiento_clase. La solución aplicada consiste en modificar trg_no_solapamiento_clase para que solo ejecute las comprobaciones de solapamiento cuando cambian campos de horario, sala o monitor (no cuando el UPDATE afecta únicamente a plazas_libres). Tras aplicar esta corrección, las 20 inscripciones se insertan correctamente y el campo plazas_libres se decrementa de forma automática.

Los datos cubren casos límite relevantes: socios sin inscripciones (consulta 5), clases con pocas plazas libres (consulta 6), contratos de distintos tipos para estadísticas de facturación (consultas 3 y 4), y socios con contratos cercanos al vencimiento (consulta 10 y cursor de alertas).

6. Batería de consultas SQL (consultas.sql)

Se han diseñado 10 consultas que cubren los requisitos del proyecto. Todas incluyen comentario de negocio en el fichero .sql.

N.º	Técnica SQL	Uso de negocio	Clave del código
1	JOIN múltiple (CLASE + SALA + MONITOR)	Listado de clases activas con sala, monitor y plazas libres. Panel de recepción.	SELECT ... FROM clase JOIN sala ... JOIN monitor ... WHERE activa='S'
2	GROUP BY + COUNT + LEFT JOIN	Número de contratos activos por tarifa. El gerente ve qué planes son más populares.	SELECT t.nombre, COUNT(c.id_contrato) ... GROUP BY t.id_tarifa ...
3	GROUP BY + HAVING + SUM (JOIN triple)	Socios que han gastado más de 200 €. Identifica clientes de alto valor.	... SUM(p.importe) ... HAVING SUM(p.importe) > 200
4	GROUP BY + MIN/MAX/AVG (pago)	Estadísticas de facturación por método de pago. Análisis para contabilidad.	... MIN, MAX, AVG, SUM ... GROUP BY p.metodo_pago
5	Subconsulta EXISTS / NOT EXISTS	Socios con contrato activo sin ninguna inscripción. Candidatos a venta cruzada. Requiere prefijo de esquema (ALTER SESSION o IRONLINKS.) para ejecutarse correctamente.	WHERE EXISTS (contrato activo) AND NOT EXISTS (inscripción)
6	JOIN + expresión derivada + filtro porcentaje	Clases con menos del 30 % de plazas libres. Alerta de aforo.	WHERE (plazas_libres / plazas_max) < 0.30
7	GROUP BY + HAVING (carga monitor)	Monitores con más de una clase activa. Control de carga de trabajo.	... COUNT(c.id_clase) ... HAVING COUNT > 1
8	Subconsulta correlacionada (MAX contrato)	Socios en tarifa Básica con más de 1 año. Candidatos a upgrade.	WHERE id_contrato = (SELECT MAX(id_contrato) FROM contrato WHERE id_socio=s.id_socio)
9	JOIN triple + CASE dentro de SUM	Ranking de clases por asistencias confirmadas. Mide popularidad real.	SUM(CASE WHEN asistio='S' THEN 1 ELSE 0 END)
10	Vista en línea (subconsulta en FROM) + RANK()	Contratos que vencen en 30 días. Recepción llama para ofrecer renovación.	RANK() OVER (PARTITION BY id_socio ORDER BY fecha_fin DESC)

7. PL/SQL: procedimientos, funciones y cursores (plsql.sql)

7.1 Procedimiento: inscribir_socio_en_clase

Parámetros: p_id_socio IN, p_id_clase IN, p_resultado OUT VARCHAR2

Utilidad de negocio: gestiona la inscripción de un socio a una clase de forma segura. Antes de insertar verifica: (1) que el socio tiene contrato activo y vigente, (2) que la clase existe y está activa, (3) que hay plazas disponibles, (4) que el socio no estaba ya inscrito. En cada comprobación devuelve un mensaje descriptivo en p_resultado. Si todo es correcto, inserta en INSCRIPCION_CLASE y hace COMMIT. El trigger trg_plazas_inscripcion descuenta la plaza automáticamente.

Estructuras PL/SQL usadas: IF/RETURN, EXCEPTION WHEN NO_DATA_FOUND, COMMIT, ROLLBACK.

7.2 Procedimiento: renovar_contrato

Parámetros: p_id_socio IN, p_id_tarifa IN, p_metodo_pago IN, p_id_contrato_n OUT, p_resultado OUT

Utilidad de negocio: cuando un abono vence, recepción puede llamar a este procedimiento para: (1) verificar el estado del socio, (2) cerrar los contratos anteriores activos (UPDATE estado='VENCIDO'), (3) crear el nuevo contrato (el trigger calcula la fecha_fin), (4) registrar el pago correspondiente, (5) reactivar el socio si estaba SUSPENDIDO. Devuelve el id del nuevo contrato y un mensaje de confirmación.

Estructuras PL/SQL usadas: múltiples UPDATE/INSERT, manejo de excepciones, COMMIT/ROLLBACK.

7.3 Función: calcular_ingresos_mes

Signatura: calcular_ingresos_mes(p_anio NUMBER, p_mes NUMBER) RETURN NUMBER

Utilidad de negocio: devuelve el total recaudado en un mes y año dados consultando la tabla PAGO_CUOTA con EXTRACT. Útil para informes mensuales de dirección y para el bloque del cursor (muestra los últimos 3 meses con un WHILE).

7.4 Función: plazas_ocupadas_clase

Signatura: plazas_ocupadas_clase(p_id_clase NUMBER) RETURN NUMBER

Utilidad de negocio: cuenta las inscripciones con estado 'P' (pendiente) o 'S' (asistido), que son las que ocupan plaza efectivamente. Devuelve -1 si la clase no existe. Complementa el campo plazas_libres de CLASE para verificaciones puntuales.

7.5 Cursor explícito: alertas de renovación

Nombre del cursor: cur_vencimientos. Selecciona contratos activos con fecha_fin entre SYSDATE y SYSDATE+15 (próximos 15 días). Para cada fila aplica lógica condicional (IF: marca como urgente si quedan 3 días o menos) y construye el mensaje de aviso con CASE.

Tras el LOOP principal incluye un bloque WHILE que recorre los 3 meses anteriores e imprime los ingresos de cada uno usando la función calcular_ingresos_mes.

Estructuras usadas: CURSOR ... IS, OPEN, FETCH, EXIT WHEN %NOTFOUND, CLOSE, LOOP, WHILE, IF, CASE.

8. Triggers (triggers.sql)

Se han implementado 6 triggers que automatizan reglas de negocio reales. Cada uno resuelve un problema que no puede cubrirse con restricciones CHECK estándar o que debe ser transparente para cualquier herramienta que acceda a la BD.

1. trg_calcular_fecha_fin

Evento: BEFORE INSERT ON contrato

Justificación: la fecha de fin de un contrato debe ser siempre `fecha_inicio + duracion_dias` de la tarifa elegida. Si dependiera de la aplicación cliente, un descuido del operador dejaría contratos sin `fecha_fin` o con fechas incorrectas. El trigger lee la duración de la tarifa y sobrescribe `fecha_fin` en `:NEW`, garantizando coherencia independientemente de cómo se realice el INSERT.

Clave del código:

Leer `duracion_dias` de `TARIFA` y asignar `:NEW.fecha_fin := :NEW.fecha_inicio + v_dias`

2. trg_plazas_inscripcion

Evento: AFTER INSERT ON inscripcion_clase

Justificación: el campo `plazas_libres` en `CLASE` debe decrementarse cada vez que un socio se inscribe. Mantener este contador en la tabla (en lugar de calcular siempre con COUNT) mejora el rendimiento de las consultas de disponibilidad. El trigger además lanza `RAISE_APPLICATION_ERROR` si no hay plazas, evitando inscripciones sobre aforo aunque la comprobación de la aplicación falle.

Clave del código:

`UPDATE clase SET plazas_libres = plazas_libres - 1 WHERE id_clase = :NEW.id_clase`

3. trg_plazas_baja

Evento: AFTER DELETE ON inscripcion_clase

Justificación: complemento del trigger anterior. Cuando un socio cancela su inscripción, la plaza debe quedar libre para que otro socio pueda ocuparla. Sin este trigger habría plazas bloqueadas fantasma. El CHECK `plazas_libres < plazas_max` evita superar el aforo original por errores de datos previos.

Clave del código:

`UPDATE clase SET plazas_libres = plazas_libres + 1 WHERE id_clase = :OLD.id_clase AND plazas_libres < plazas_max`

4. trg_auditoria_socios

Evento: AFTER INSERT OR UPDATE OR DELETE ON socio

Justificación: el RGPD exige trazabilidad de los cambios en datos personales. Este trigger registra altas (INSERT), cambios de estado o email (UPDATE) y bajas (DELETE) en la tabla AUDITORIA_SOCIOS con usuario de BD, fecha exacta y valores anterior y posterior. Al operar a nivel de BD, protege incluso frente a accesos directos por SQL*Plus o herramientas de administración.

Clave del código:

```
INSERT INTO auditoria_socios (...) VALUES (...accion, campo, valor_antes, valor_despues...)
```

5. trg_validar_edad_socio

Evento: BEFORE INSERT OR UPDATE ON socio

Justificación: el gimnasio no admite socios menores de 16 años sin autorización especial. Una restricción CHECK no puede comparar fecha_nac con SYSDATE porque SYSDATE es dinámica. El trigger calcula la edad real con MONTHS_BETWEEN / 12 y lanza un error si es inferior a 16, con mensaje informativo que incluye la edad calculada.

Clave del código:

```
IF FLOOR(MONTHS_BETWEEN(SYSDATE, :NEW.fecha_nac)/12) < 16 THEN  
RAISE_APPLICATION_ERROR(-20002,...)
```

6. trg_no_solapamiento_clase

Evento: BEFORE INSERT OR UPDATE ON clase

Justificación: un monitor no puede impartir dos clases a la vez, y una sala no puede albergar dos clases simultáneas. Estas restricciones implican comparar la fila nueva con otras filas de la misma tabla, lo que es imposible con CHECK.

Corrección aplicada: el trigger original provocaba ORA-04091 (tabla mutante) al encadenarse con trg_plazas_inscripcion. El motivo era que el UPDATE de plazas_libres disparaba trg_no_solapamiento_clase, que a su vez intentaba hacer un SELECT sobre CLASE mientras esta estaba en proceso de modificación. La solución consiste en añadir una condición IF que hace que el trigger solo ejecute las comprobaciones de solapamiento cuando cambian id_monitor, id_sala, dia_semana u hora_inicio, ignorando los UPDATE que solo afectan a plazas_libres.

Clave del código:

```
IF INSERTING OR (UPDATING AND (:NEW.id_monitor != :OLD.id_monitor OR  
:NEW.id_sala != :OLD.id_sala OR :NEW.dia_semana != :OLD.dia_semana OR  
:NEW.hora_inicio != :OLD.hora_inicio)) THEN ... END IF;
```

9. Conclusión

El proyecto Iron Links Gym demuestra cómo una base de datos Oracle bien diseñada puede cubrir el ciclo de vida completo de un negocio real: desde el alta de un socio hasta la renovación de su abono, pasando por la gestión de clases, el control de aforo y el registro de pagos.

Las decisiones de diseño (claves sintéticas, separación de TARIFA y CONTRATO, tabla de auditoría sin FK para preservar el histórico, constraints UNIQUE en email y teléfono de MONITOR y en email de SOCIO) están motivadas por requisitos de negocio concretos, no por convención.

Los triggers y procedimientos implementados resuelven problemas que no podrían resolverse únicamente con restricciones declarativas. En particular, la corrección del trigger `trg_no_solapamiento_clase` para evitar el error ORA-04091 ilustra la importancia de entender la cadena de disparadores y el concepto de tabla mutante en Oracle.

El código SQL y PL/SQL se ha redactado con nombres descriptivos, indentación consistente y comentarios orientados al negocio, de forma que cualquier persona del equipo pueda mantenerlo sin necesidad de conocer el contexto original del desarrollo.

Dan Bolocan · CPIFP Los Enlaces · 14/05/2026